



Stack



Algoritma & Struktur Data



Stack

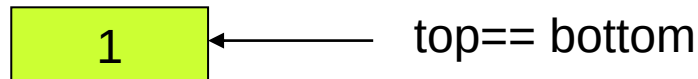
- ▶ Stack adalah list linier yang:
 - ▶ Dikenali elemen puncaknya (TOP)
 - ▶ Aturan penyisipan dan penghapusan yaitu:
 - ▶ Penyisipan selalu dilakukan “di atas” TOP
 - ▶ Penghapusan selalu dilakukan pada TOP
 - ▶ TOP adalah satu-satunya alamat tempat terjadi operasi
 - ▶ Elemen stack tersusun secara LIFO (Last In First Out)



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling atas
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **PUSH** : Menambahkan elemen pada sebuah stack

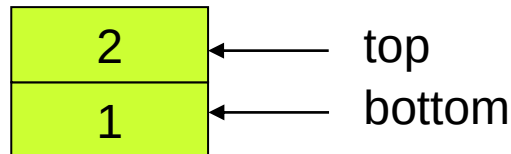
PUSH



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **PUSH** : Menambahkan elemen pada sebuah stack

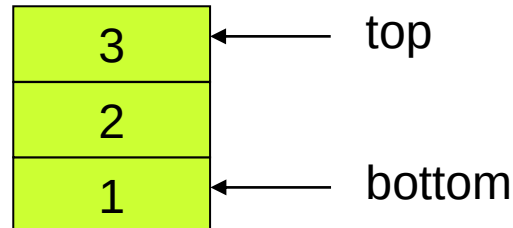
PUSH



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **PUSH** : Menambahkan elemen pada sebuah stack

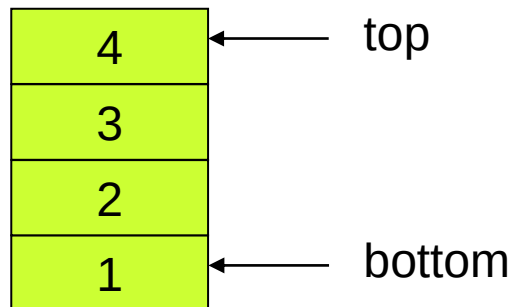
PUSH



Apakah Stack itu ?

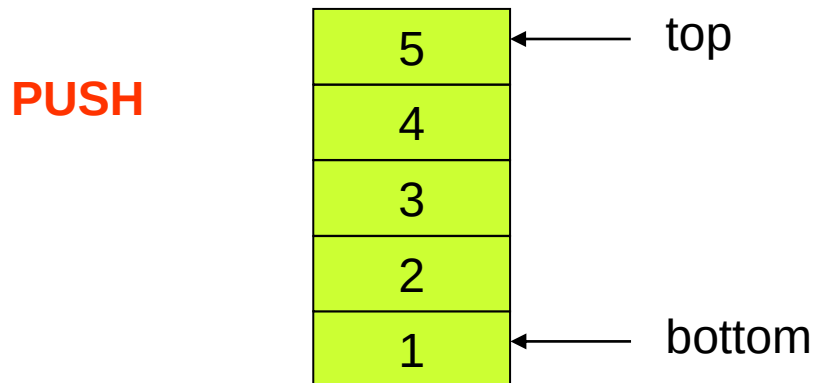
- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **PUSH** : Menambahkan elemen pada sebuah stack

PUSH



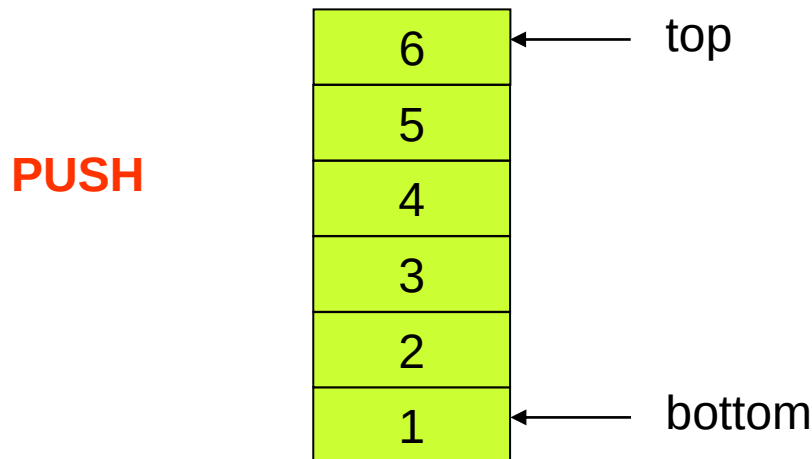
Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **PUSH** : Menambahkan elemen pada sebuah stack



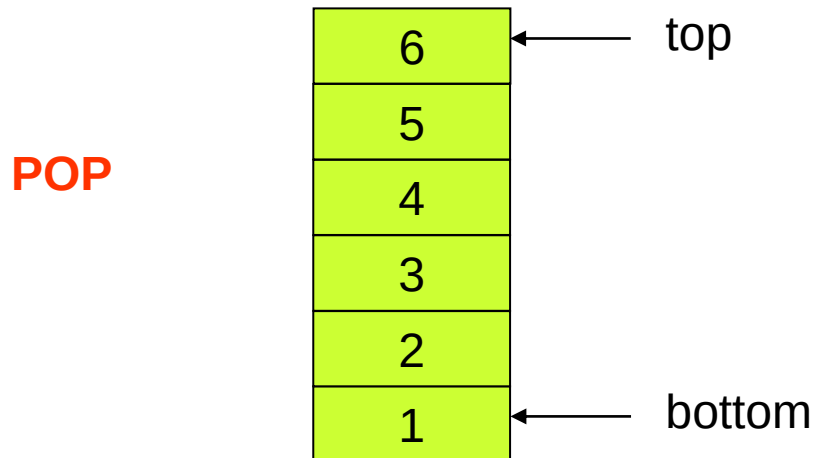
Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **PUSH** : Menambahkan elemen pada sebuah stack



Apakah Stack itu ?

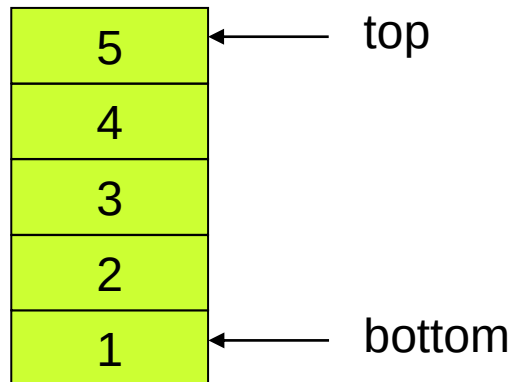
- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **POP** : Menghapus sebuah elemen dari sebuah stack



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **POP** : Menghapus sebuah elemen dari sebuah stack

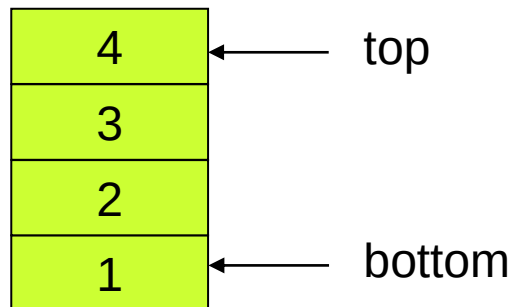
POP



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **POP** : Menghapus sebuah elemen dari sebuah stack

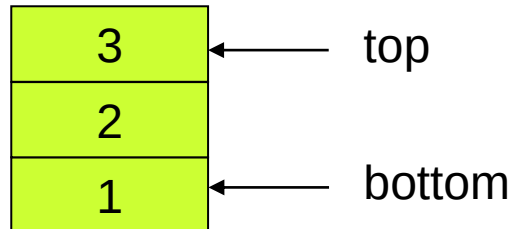
POP



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **POP** : Menghapus sebuah elemen dari sebuah stack

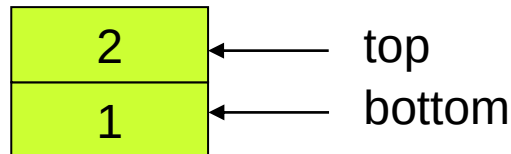
POP



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **POP** : Menghapus sebuah elemen dari sebuah stack

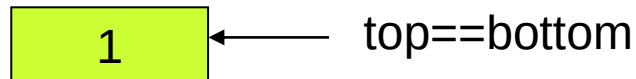
POP



Apakah Stack itu ?

- ▶ Penambahan dan penghapusan elemen dilakukan pada elemen list yang terletak di paling depan
- ▶ Yang dihapus adalah elemen yang paling terakhir ditambahkan
- ▶ Nama lain: LIFO (Last In First Out)
- ▶ Operasi **POP** : Menghapus sebuah elemen dari sebuah stack

POP



Stack Overflow & Stack Underflow

- ▶ Stack Overflow

Menambahkan data pada sebuah stack yang telah penuh

- ▶ Stack Underflow

Menghapus data dari sebuah stack yang sudah kosong



Operasi yang Dapat Dilakukan pada Sebuah Stack

1. ***createstack***

- ▶ membuat tumpukan baru, tentu saja mulai dari kosong

2. ***isemptystack***

- ▶ memeriksa apakah stacknya kosong

3. ***isfullstack***

- ▶ Memeriksa apakah stack penuh

4. ***clearstack***

- ▶ Mengeluarkan seluruh isi stack

5. ***push***

- ▶ Memasukan sebuah piring kedalam stack, tentunya jika stack tidak penuh

6. ***pop***

- ▶ Mengambil sebuah piring dari stack, tentunya jika stack tidak kosong



Implementasi Stack pada Program

- ▶ Stack adalah *abstract data type* (ADT), maka implementasinya dipisahkan dari *interface/representasinya*.
- ▶ Pengguna hanya mengoperasikan stack sesuai dengan *interfacenya*.
- ▶ Ada tiga hal yang perlu diperhatikan pada sebuah stack, yaitu:
 - ▶ Tempat untuk tumpukan **data** (bisa berupa array static, dynamic, atau linked list)
 - ▶ Maksimal data yang bisa ditumpuk (disebut ukuran atau **size** dari stack), dan
 - ▶ Jumlah data yang sudah ada pada tumpukan saat ini (disebut **top**)



-
- ▶ Stack dapat diimplementasikan dengan berbagai cara, diantaranya:
 - ▶ Implementasi stack dengan **static array**
 - ▶ Implementasi stack dengan **dynamic array**
 - ▶ Implementasi stack dengan **linked list**



Implementasi Stack dengan Static Array

```
struct tstack {  
    int data[1000];  
    int size;  
    int top;  
} ;
```

Untuk memudahkan bisa ditulis sebagai berikut :

```
typedef struct {  
    int data[1000], size, top;  
} tstack;
```



Membuat Stack Kosong

```
tstack createstack(int ukuran) {  
    tstack s;  
    s.size = ukuran;  
    s.top = -1;  
    return s;  
}
```



Memeriksa Stack Kosong dan Penuh

```
int isemptystack(tstack s) {  
    if (s.top == -1) return 1;  
    return 0;  
}
```

```
int isfullstack(tstack s) {  
    if ((s.top + 1) == s.size) return 1;  
    return 0;  
}
```



Operasi Penambahan Elemen Stack

```
int push(tstack *s, int data) {  
    if (isfullstack(*s)) return 0;  
    (*s).top++;  
    (*s).data[(*s).top] = data;  
    return 1;  
}
```



Operasi Pengeluaran Elemen Stack

```
int pop(tstack *s, int *hasil) {  
    if (!isemptystack(*s)) {  
        *hasil = (*s).data[(*s).top];  
        (*s).top--;  
        return 1;  
    }  
    return 0;  
}
```



Latihan

- ▶ **Buatlah implementasi Stack dengan**
 - ▶ Dynamic Array
 - ▶ Linked List
- ▶ **Adapun operasi yang perlu dibuat:**
 - ▶ Create Stack
 - ▶ Is Empty & Full Stack
 - ▶ Push
 - ▶ Pop

